Практика | Задание №16, 23 Часть 2

Уровень: "Важный"

Задание №1

Алгоритм вычисления значения функции F(n) и G(n), где n – целое число, задан следующими соотношениями:
F(n)=3·G(n−3)+11
G(n)=5·n, при n<12
G(n)=G(n−4)+7, при n>=12
Чему равно значение выражения F(20001)?

Верные варианты

105098

Решение

Сообщить об ошибке

Напишем программу на языке Python.
Решение через функции.
Импортируем библиотеку и декоратор для кэширования:
```
from functools import lru_cache
```
Кэшируем с помощью @lru_cache(None):
```
@lru_cache(None)
```
Создаем функцию G(n):
```
def G(n):
  if n < 12:
    return 5 * n
  return G(n - 4) + 7
@lru_cache(None)
```
Создаем функцию для F(n) и возвращаем значение функции из условия:
```
def F(n):
  return 3 * G(n - 3) + 11
```
Перебор для того, чтобы все значения кешировались для ускорения работы программы:
```
for i in range(0, 50000):
  F(i)
  G(i)
```
Выводим ответ:
```
print(F(20001))
```

Полный код:
```
from functools import lru_cache
@lru_cache(None)
def G(n):
  if n < 12:
    return 5 * n
  return G(n - 4) + 7
@lru_cache(None)
def F(n):
  return 3 * G(n - 3) + 11
for i in range(0, 50000):
  F(i)
  G(i)
print(F(20001))
```

INSANEK.RU

Решение через списки
Создадим переменную для максимальной длины массивов:
```
N = 50000
G_vals = [0] * (N + 1)
F_vals = [0] * (N + 1)
```
Переберем все аргументы функции и заполним массив G по условию:
```
for n in range(N + 1):
  if n < 12:
    G_vals[n] = 5 * n
  else:
```

```
    G_vals[n] = G_vals[n - 4] + 7
```

Переберем все аргументы функции и заполним массив F по условию:

```
for n in range(3, N + 1):
  F_vals[n] = 3 * G_vals[n - 3] + 11
```

Выводим ответ:

```
print(F_vals[20001])
```

Полный код

```
N = 50000
G_vals = [0] * (N + 1)
F_vals = [0] * (N + 1)
for n in range(N + 1):
  if n < 12:
    G_vals[n] = 5 * n
  else:
    G_vals[n] = G_vals[n - 4] + 7
for n in range(3, N + 1):
  F_vals[n] = 3 * G_vals[n - 3] + 11
print(F_vals[20001])
```

Ответ: 105098.

Задание №2

Алгоритм вычисления значения функции F(n) и G(n), где n – целое число, задан следующими соотношениями:
F(n)=2·(G(n−2)+9)
G(n)=2·n, при n < 20
G(n)=G(n−2)+3, при n >= 20
Чему равно значение выражения F(16888)?

Верные варианты

50694

Решение

Сообщить об ошибке

Напишем программу на языке Python.
Решение через функции.
Импортируем библиотеку для кэширования результатов функций:

```
from functools import lru_cache
```

Включаем кэширование для функции F:

```
@lru_cache(None)
def F(n):
```

Возвращаем значение функции:

```
  return 2 * (G(n - 2) + 9)
```

Включаем кэширование и для функции G, если n<20, возвращаем n*2, иначе G(n-2)+3:

```
@lru_cache(None)
def G(n):
  if n < 20:
    return 2 * n
  return G(n - 2) + 3
```

Вычисляем F(i) для всех значений от 1 до 49999, чтобы кэшировать и ускорить последующие вызовы:

INSANEK.RU

```
for i in range(1, 50000):
  F(i)
```

Выводим результат для n = 16888:

```
print(F(16888))
```

Полный код:

```
from functools import lru_cache
@lru_cache(None)
def F(n):
  return 2 * (G(n - 2) + 9)
@lru_cache(None)
def G(n):
  if n < 20:
    return 2 * n
  return G(n - 2) + 3
```

```
for i in range(1, 50000):
   F(i)
print(F(16888))
```

Решение через списки

Максимальное значение, до которого будем считать:

N = 50000

Создадим список для значений F и G (от 0 до N)

F = [0] * (N + 1)

G = [0] * (N + 1)

Сначала вычисляем G по функции из условия:

```
for n in range(N + 1):
   if n < 20:
      G[n] = 2 * n
   else:
      G[n] = G[n - 2] + 3
```

Теперь вычисляем F по функции из условия:

```
F = [0] * (N + 1)
G = [0] * (N + 1)
for n in range(N + 1):
   if n >= 2:
      F[n] = 2 * (G[n - 2] + 9)
   else:
      F[n] = 0
```

Вывод результата:

```
print(F[16888])
```

Полный код

```
for n in range(N + 1):
   if n < 20:
      G[n] = 2 * n
   else:
      G[n] = G[n - 2] + 3

for n in range(N + 1):
   if n >= 2:
      F[n] = 2 * (G[n - 2] + 9)
   else:
      F[n] = 0
print(F[16888])
```

Ответ 50694.

Задание №3

Тело функции F(n) задано следующими строчками:

$F(n) = n^3$, при $n \le 3$,

$F(n) = n + 2 + 8 * F(n - 2)$, при чётном n и если $n > 3$,

$F(n) = 17 * n + F(n - 3) - F(n - 4) + 8$, при нечётном n и если $n > 3$.

При условии, что F(n) кратно шести, найдите количество таких значений n из диапазона [134; 928], при которых данное условие истинно.

Верные варианты

199

INSANEK.RU

Решение

Сообщить об ошибке

Ответ: 199

Объявим функцию def F(n) и перепишем условие.

```
def F(n):
   if n <= 3:
      return n * n * n
   if n > 3 and n % 2 == 0:
      return n + 2 + 8 * F(n - 2)
   if n > 3 and n % 2 != 0:
```

```
        return 17 * n + F(n - 3) - F(n - 4) + 8
cnt = 0 #создаем переменную-счетчик
for n in range(134, 928 + 1): #создаем цикл перебора n с диапазоном, указанном в условии
    if F(n) % 6 == 0: #проверяем кратность шести и увеличиваем счётчик
        cnt += 1
print(cnt) #печатаем результат счетчика
```

Задание №4

У исполнителя есть две команды:
— прибавить 1;
— умножить на 3.
Первая из них увеличивает число на 1, вторая увеличивает число в 3 раза. Сколько существует программ, которые преобразуют исходное число 1 в число 21 и не содержат в траектории вычислений число 5?

Верные варианты

7

Решение

Сообщить об ошибке

Решаем с помощью языка Python:
1) Для того, чтобы эффективнее решить данную задачу создадим рекурсивную функцию, которая будет считать количество возможных программ.
```
def task23(start, end):
```
2) В случае, если число старта становится больше числа конца, необходимо возвратить ноль, если же равно, то нужно возвратить единицу. Также, если число старта станет равно 5, то тоже возвращается ноль:
```
    if start > end or start == 5:
        return 0
    elif start == end:
        return 1
```
3) Во всех остальных случаях возвращается выражение с нашей функцией, в которой мы будем прибавлять единицу от старта или умножать его на 3.
```
    else:
        return task23(start + 1, end) + task23(start * 3, end)
```
4) Выводим количество результатов функции от 1 до 21.
```
print(task23(1, 21))
```

Код программы:
```
def task23(start, end):
    if start > end or start == 5:
        return 0
    elif start == end:
        return 1
    else:
        return task23(start + 1, end) + task23(start * 3, end)
print(task23(1, 21))
```
Решение руками:
Поставим начальному числу значение 1. Для начального и каждого последующего числа используем следующий алгоритм: к значению чисел, которые можно получить из текущего, будем прибавлять значение текущего. Избегаемому числу необходимо поставить значение 0, чтобы не учитывать проходящие через него программы.

INSANEK.RU

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	1	+1	2	0	1	1	1	+2	3	3	+2	5	5	5	5	5	+1	6	6	+1
		+1						+1			+3						+5			+6
		2						3			5						6			7

Задание №5

Есть исполнитель, который преобразует двоичное число на экране. У него есть две команды:
1. Прибавить 1 (увеличивает число на 1).
2. Приписать справа двоичную запись остатка от деления текущего числа в десятичной системе счисления на 8 (добавляет к числу справа

двоичную запись остатка от деления на 8).
Найдите количество программ, которые преобразуют число 1_2 в 10111110_2.
Например, траектория 121 для двоичного числа 3 будет содержать следующие двоичные числа: 100_2, 100100_2, 100101_2.
В ответе запишите одно целое число — найденное количество программ.

Верные варианты

1375

Решение

Сообщить об ошибке

1) Для того чтобы эффективнее решить данную задачу, создадим рекурсивную функцию, которая будет считать количество возможных программ.
2) В случае, если число старта становится больше числа конца возвратим ноль, если же равно, то единицу.
3) Во всех остальных случаях возвращается выражение с нашей функцией, в которой мы будем прибавлять единицу к старту и приписывать двоичную запись остатка от деления текущего числа на 8.
Полный код программы:

```
def task23(start, end):
   if start > end:
      return 0
   if start == end:
      return 1
   return task23(start + 1, end) + \
    task23(int(bin(start)[2:] + bin(start % 8)[2:], 2), end)
print(task23(1, int("10111110", 2)))
```

Получаем ответ: 1375.

Уровень: "Сложный"

Задание №6

Алгоритм вычисления значения функции F(m, n), где m и n – целые неотрицательные числа, задан следующими соотношениями:
F(0, n) = n + 1;
F(m, 0) = F(m - 1, 1), если m > 0;
F(m, n) = F(m - 1, F(m, n - 1)), если m > 0 и n > 0.
Определите и запишите в ответ значение F(m,n), если в нее передать числа m = 4 и n = 1.

Верные варианты

65533

Решение

Сообщить об ошибке

1. Стандартным способом решить не получится, так как программа при нужных значениях выдает ошибку о глубине рекурсии. Значит решим его через создание списка, который заполним всеми необходимыми значениями.
2. Создадим двумерный список через цикл. Если внимательнее приглядеться к 3 строке алгоритма, то можно понять, что при m > 10, значения будут слишком большими. Значит нет смысла создавать список длины > 10.
3. При помощи цикла добавим в каждый элемент F по 1000000 нулей. Так нужно, потому что в 3 строке алгоритма мы обращаемся по F[m][n-1] индексу. Значение может быть очень большим, а значит и количество n надо взять с запасом.
4. Теперь вложенным циклом перебираем m и n. Переписываем код из алгоритма, обращаясь к F(m, n) через индексы.
5. Выводим F[4][1] – ответ.

INSANEK.RU

Код:

```
F = []
for i in range(10):
   F.append([0] * 1000000)
for m in range(10):
   for n in range(500000):
      if m == 0:
         F[m][n] = n + 1
      if m > 0 and n == 0:
         F[m][n] = F[m-1][1]
      if m > 0 and n > 0:
         F[m][n] = F[m-1][F[m][n-1]]
```

```
print(F[4][1])
```

Задание №7

Исполнитель преобразует число, записанное на экране. У исполнителя есть три команды, которым присвоены номера:
1. Умножить на 3
2. Умножить на 2
3. Прибавить 2
4. Прибавить 1
Первая команда увеличивает значение числа в 3 раза, вторая – в 2, третья – увеличивает значение числа на 2, четвертая - на 1. Программа для исполнителя – последовательность команд.
Сколько существует таких программ, которые исходное число 2 преобразуют в число 25, при этом в программе номера команд идут в нестрогом порядке возрастания?

Верные варианты

59

Решение

Сообщить об ошибке

1. Основная идея решения в том, чтобы сохранять каждую команду и проверять в возрастающем ли она порядке. Удобнее всего создать третий аргумент – пустую строку и заполнять ее номерами команд.
2. Пишем стандартный код. Если первый аргумент больше второго, значит получили больше 25, такой вариант не подходит, возвращаем 0. Если первый аргумент равен второму (25 == 25), тогда проверяем идут ли номера команд в порядке возрастания. Удобно отсортировать строку при помощи sorted(), а потом склеить ее вновь, используя join().
3. Если первый аргумент меньше второго, значит расписываем все возможные действия, “приклеивая” к третьему аргументу номера команд.
4. Во всех остальных случаях возвращаем 0, они нам точно не подходят.
Код:

```
def F(start,end,com):
    if start > end:
        return 0
    if start == end and ''.join(sorted(com)) == com:
        return 1
    if start < end:
        return F(start * 3, end, com + '1') + F(start * 2, end, com + '2') + F(start + 2, end, com + '3') + F(start + 1, end, com + '4')
    else:
        return 0
print(F(2,25,''))
```

Уровень: "Нет данных"

Задание №8

Функции F(n) и G(n) заданы следующими строчками:
F(3) = G(3) = 3,
F(n) = 4 · F(n - 1) + 4 · G(n - 1), при n > 3,
G(n) = n + 2 · F(n - 1) + 2 · G(n - 1), при n > 3.
Определите и запишите в ответ, что возвратит функция G, если в неё передать аргумент n = 9?

INSANEK.RU

Верные варианты

106381

Решение

Сообщить об ошибке

Для решения напишем программу на языке Python.
1. Первым делом нужно объявить функцию F(n).
2. В функции пропишем два условия:

- При n = 3:
  if n == 3:
  return 3
- При n > 3:
  if n > 3:
  return 4 * F(n - 1) + 4 * G(n - 1)

3. Теперь точно так же пропишем и для функции G(n). Объявляем функцию.
4. В функции пропишем два условия:
   - При n = 3:
     if n == 3:
     return 3
   - При n > 3:
     if n > 3:
     return n + 2 * F(n - 1) + 2 * G(n - 1)

5. Теперь остаётся только вывести значение G(9) и записать его в ответ.

Полный код:
```
def F(n):
  if n == 3:
    return 3
  if n > 3:
    return 4 * F(n - 1) + 4 * G(n - 1)
def G(n):
  if n == 3:
    return 3
  if n > 3:
    return n + 2 * F(n - 1) + 2 * G(n - 1)
print(G(9))
```

Задание №9

Пусть [x] – это целая часть числа x, а / – обычное, нецелочисленное деление. Функция F(n) задана следующими строчками и условиями:
F(1) = 2,
F(2) = 3,
F(n) = [(F(n - 2) + F(n - 2)) / 7], при нечетном n и n > 2,
F(n) = 7 · n - [F(n - 1) / 2 + 5], при четном n и n > 2.
При этом n — целое неотрицательное число. Определите, какое значение вернет данная функция, если в неё передать аргумент n = 40.

Верные варианты

275

Решение

Сообщить об ошибке

Эффективнее всего решить это задание программированием, реализовав алгоритм через рекурсивную функцию на Python.
1) Сначала нам потребуется объявить функцию.
def F(n):
2) Реализуем условие задания с помощью четырех условных операторов if. Неравенства "при n > 2" будем записывать как "if n > 2:", условие, например, нечетности будет проверяться как "n % 2 != 0", а целую часть числа будем находить с помощью функции int(). Также будем использовать return для реализации рекурсивности функции.

UNSANEK.RU

```
  if n == 1:
    return 2
  if n == 2:
    return 3
  if n > 2 and n % 2 != 0:
    return int((F(n - 2) + F(n - 2)) / 7)
  if n > 2 and n % 2 == 0:
    return 7 * n - int(F(n-1) / 2 + 5)
```
3) Последним шагом будем выводить на печать возвращаемое функцией F(n) значение при заданном аргументе n=40.
print(F(40))
Код программы:
```
def F(n):
  if n == 1:
```

```
    return 2
  if n == 2:
    return 3
  if n > 2 and n % 2 != 0:
    return int((F(n - 2) + F(n - 2)) / 7)
  if n > 2 and n % 2 == 0:
    return 7 * n - int(F(n-1) / 2 + 5)
print(F(40))
```

Задание №10

Тело функции F(n) задано следующими строчками:
F(n) = n * n * n + 22 , при n <= 14,
F(n) = F(n - 4) + F(n - 8), при кратном трём n и если n > 14,
F(n) = F(n - 9) + n + 7, при не кратном трём n и если n > 14.
При условии, что запись значения F(n) не содержит нечётных цифр, найдите количество таких значений n из диапазона [1; 199], при которых данное условие истинно.

Верные варианты

16

Решение

Сообщить об ошибке

Для решения напишем программу на языке Python.
1. Первым делом нужно объявить функцию F(n).
2. В функции пропишем три условия:
   - **При n <= 14:**
     if n <= 14:
     return n * n * n + 22
   - **При n > 14, кратном 3:**
     if n % 3 == 0 and n > 14:
     return F(n - 4) + F(n - 8)
   - **При n > 14, не кратном 3:**
     if n % 3 != 0 and n > 14:
     return F(n - 9) + n + 7

3. Создадим переменную count, в которой будет храниться количество подходящих значений:
count = 0
4. Создадим цикл for с границами 1 и 200, так как последнее значение не включается:
for i in range(1, 200):
5. Проверяем каждую нечётную цифру по отдельности, чтобы её не было в строковом представлении значения функции F. Если условие выполняется, увеличиваем счётчик на единицу:
if "1" not in str(F(i)) and "3" not in str(F(i)) and "5" not in str(F(i)) and "7" not in str(F(i)) and "9" not in str(F(i)) :
count += 1
6. Остаётся вывести количество:
print(count)

Полный код:

```
def F(n):
  if n <= 14:
    return n * n * n + 22
  if n % 3 == 0 and n > 14:
    return F(n - 4) + F(n - 8)
  if n % 3 != 0 and n > 14:
    return F(n - 9) + n + 7
count = 0
for i in range(1, 200):
  if "1" not in str(F(i)) and "3" not in str(F(i)) and "5" not in str(F(i)) and "7" not in str(F(i)) and "9" not in str(F(i)) :
    count += 1
print(count)
```

INSANEK.RU

Задание №11

У исполнителя есть две команды:
– вычесть 1
– разделить нацело на 2
Определите количество программ исполнителя, которые преобразуют число 43 в 1, при условии, что траектория выполнения программы содержит число 10. В ответ запишите целое число – количество программ.

Правильный ответ:

9 3 0

Решение

Сообщить об ошибке

Решаем с помощью языка Python:
1) Для того, чтобы эффективнее решить данную задачу создадим рекурсивную функцию, которая будет считать количество возможных программ.
def task23(start, end):
2) В случае, если число старта становится меньше числа конца, необходимо возвратить ноль, если же равно, то нужно возвратить единицу.

```
  if start < end:
    return 0
  elif start == end:
    return 1
```

3) Во всех остальных случаях возвращается выражение с нашей функцией, в которой мы будем вычитать единицу от старта или делить нацело его на 2.

```
  else:
    return task23(start-1, end) + task23(start//2, end)
```

4) Так как число 10 должно входить в траекторию, перемножим количество программ от 43 до 10 с количеством программ от 10 до 1.
print(task23(43, 10) * task23(10, 1))

Код программы:

```
def task23(start, end):
  if start < end:
    return 0
  elif start == end:
    return 1
  else:
    return task23(start-1, end) + task23(start//2, end)
print(task23(43, 10) * task23(10, 1))
```

Задание №12

У исполнителя есть две команды:
– прибавить 1
– умножить на 2
Определите количество программ исполнителя, которые преобразуют число 4 в 93, при условии, что траектория выполнения программы содержит число 66 и не содержит число 23. В ответ запишите целое число – количество программ.

Верные варианты

225

INSANEK.RU

Решение

Сообщить об ошибке

Решаем с помощью языка Python:
1) Для того, чтобы эффективнее решить данную задачу, создадим рекурсивную функцию, которая будет считать количество возможных программ.
def task23(start, end):
2) В случае, если число старта становится больше числа конца, необходимо возвратить ноль, если же равно, то нужно возвратить единицу. Ещё у нас есть число, которое не должно входить в траекторию, поэтому при равенстве старта с ним мы также будем возвращать ноль.

```
  if start > end or start == 23:
    return 0
  elif start == end:
    return 1
```

3) Во всех остальных случаях возвращается выражение с нашей функцией, в которой мы будем прибавлять единицу к старту или

умножать его на 2.
```
    else:
        return task23(start + 1, end) + task23(start * 2, end)
```
4) Так как число 66 должно входить в траекторию, перемножим количество программ от 4 до 66 с количеством программ от 66 до 93.
```
print(task23(4, 66) * task23(66, 93))
```

Код программы:
```
def task23(start, end):
    if start > end or start == 23:
        return 0
    elif start == end:
        return 1
    else:
        return task23(start + 1, end) + task23(start * 2, end)
print(task23(4, 66) * task23(66, 93))
```

Задание №13

У исполнителя есть три команды:
– прибавить 1
– умножить на 3
– умножить на 4
Определите количество программ исполнителя, которые преобразуют число 3 в 70, при условии, что траектория выполнения программы содержит число 10, но не содержит 35. В ответ запишите целое число – количество программ.

Правильный ответ:

4 0

Решение

Сообщить об ошибке

Решаем с помощью языка Python:
1) Для того, чтобы эффективнее решить данную задачу создадим рекурсивную функцию, которая будет считать количество возможных программ. В качестве входных параметров передаем два значения — start (число, из которого начинаем подсчет программ) и end (число, в которое двигаемся):
```
def task23(start, end):
```
2) В случае, если число старта становится больше числа конца или число старта равно 35, необходимо возвратить ноль, если же они равны, то нужно возвратить единицу:
```
 if start > end or start == 35:
        return 0
    elif start == end:
        return 1
```
3) Во всех остальных случаях возвращается выражение с нашей функцией, в которой мы будем прибавлять единицу к старту, умножать его на 3 или умножать его на 4 (по условию задачи):
```
    else:
        return task23(start+1, end) + task23(start*3, end) + task23(start*4, end)
```
4) Так как число 10 должно входить в траекторию, перемножим количество программ от 3 до 10 с количеством программ от 10 до 70:
```
print(task23(3, 10) * task23(10, 70))
```
Код программы:
```
def task23(start, end):
    if start > end or start == 35:
        return 0
    elif start == end:
        return 1
    else:
        return task23(start+1, end) + task23(start*3, end) + task23(start*4, end)
print(task23(3, 10) * task23(10, 70))
```

INSANEK.RU